

Guida per provare Tela su Windows

Questa guida non dà per scontato niente: nessuna riga di comando, nessun programma, nessun termine tecnico che non venga spiegato prima di usarlo. Se un passaggio non torna, fermati lì — il resto della guida presuppone che quel passaggio sia andato a buon fine.

Cosa faremo: installare tre programmi (una volta sola), scaricare il progetto, scrivere una o più chiavi segrete in un file, e poi due semplici comandi ogni volta che vuoi riaprire l'app.

IN QUESTA GUIDA

- Cos'è Tela
- Prima di iniziare
- 1 Installare Node.js
- 2 Installare Python
- 3 Installare uv
- 4 Installare GitHub Desktop
- 5 Scaricare il progetto
- 6 Aprire il terminale nella cartella del progetto
- 7 Installare i "pezzi" del progetto (una volta sola)
- Provider e API key
- 8 Scrivere le chiavi
- 9 Avviare il "cervello" dell'app
- 10 Avviare l'app
- Come si usa, in breve
- Provare un provider diverso
- Le prossime volte (versione corta)
- Se qualcosa non funziona

Cos'è Tela, e dove sta andando

Tela è un armadio digitale. Fotografi i tuoi capi, l'app li legge da sola (categoria, colore, tessuto, come si lava) e li tiene in un elenco che puoi sfogliare e filtrare. Da lì fa due cose:

- **legge una foto** e capisce di che capo si tratta, senza che tu debba compilare niente a mano;
- **propone un outfit**, guardando cosa hai pulito, il meteo, e cosa non indossi da tempo — e spiega perché, non si limita a dire un nome.

Entrambe passano da un modello di intelligenza artificiale: è il pezzo di cui parla questa guida, e il motivo per cui servono le chiavi del Passo 8.

Dove sta andando: oggi il manichino che «prova» i vestiti li tinge di un colore piatto, non li indossa davvero. La direzione — scritta ma non ancora costruita — è che l'avatar indossi la **foto vera** di ogni capo, ritagliata e applicata a un corpo che assomiglia a te, non a un manichino anonimo. È il cambiamento più grande ancora da fare.

Cosa non è ancora vero, ed è meglio saperlo prima di girare l'app a qualcun altro:

- **Nessun modello è mai stato interrogato davvero.** Tutto è stato costruito e provato fino al confine — prompt scritto, chiamata pronta, errori gestiti — ma senza una chiave vera collegata. Il primo giro che fai seguendo questa guida è anche il primo collaudo reale: aspettati che qualche testo del prompt vada ritoccato dopo averlo visto rispondere.
- **Il manichino 3D non è mai stato visto girare fuori da un browser.** Va provato su un telefono o un simulatore, non solo sul web.
- **Non c'è ancora nessun account online.** Tutto quello che fai con questa guida gira sul tuo computer: non esiste un server nostro acceso da qualche parte, e i dati non si salvano da un riavvio all'altro (lo ripetiamo più avanti, perché sorprende).

Niente di tutto questo blocca la prova — è il contesto per capire cosa stai guardando.

Prima di iniziare

Chi ti ha mandato questa guida deve darti due cose, **prima** che tu arrivi al passo 8:

1. Un invito su GitHub come collaboratore del progetto (arriva una email da GitHub — vai cliccato "Accept invitation").
2. Una o più "chiavi" (stringhe di lettere e numeri), mandate in privato — per messaggio, non per email in chiaro se possibile. Non farci nulla finché non arrivi al passo 8: tienile semplicemente da parte.

1

Installare Node.js

Node.js è il programma che fa girare la parte "app" del progetto.

1. Vai su nodejs.org
2. Clicca il bottone grande a sinistra (quello con scritto "LTS" — è la versione consigliata, non quella "Current")
3. Si scarica un file tipo `node-v22.x.x-x64.msi`. Aprilo (doppio clic)
4. Segui l'installazione cliccando **Avanti** su ogni schermata, poi **Installa**, poi **Fine**. Le impostazioni di default vanno bene, non c'è nulla da cambiare.

VERIFICA CHE ABBIA FUNZIONATO:

1. Premi il tasto Windows, digita `PowerShell`, premi Invio — si apre una finestra blu (o nera) con del testo: è il **terminale**, il posto dove scriveremo dei comandi per tutta la guida.
2. Scrivi (o incolla) questo e premi Invio:

```
node -v
```

3. Deve rispondere con qualcosa tipo `v22.14.0`. Se invece dice "node non è riconosciuto...", chiudi PowerShell, riaprilo, e riprova — a volte serve un riavvio della finestra dopo un'installazione.

2

Installare Python

Python è il programma che fa girare la parte che parla con l'intelligenza artificiale.

1. Vai su python.org/downloads
2. Clicca il bottone giallo "Download Python 3.1x.x" (deve essere **3.12 o più recente** — se il sito propone una versione più vecchia, cerca nella pagina "Looking for a specific release?" e scegli una 3.12.x o 3.13.x)
3. Apri il file scaricato.

Attenzione, questo passaggio si salta facilmente e serve: nella prima schermata dell'installer, in basso, c'è una casella "Add python.exe to PATH". Spuntala **prima** di cliccare "Install Now".

5. Aspetta che finisca, poi "Close".

VERIFICA (IN UNA FINESTRA POWERSHELL NUOVA — CHIUDI QUELLA VECCHIA E RIAPRINE UNA, SEMPRE TASTO WINDOWS → SCRIVI POWERSHELL → INVIO):

```
python --version
```

Deve rispondere `Python 3.12.x` o simile. Se dice "non riconosciuto", quasi sempre è perché la casella del punto 4 non era spuntata: disinstalla Python da "Impostazioni → App", e reinstallalo rifacendo attenzione a quel passaggio.

3

Installare uv

uv è lo strumento che scarica i "pezzi" di cui Python ha bisogno per questo progetto.

1. Apri PowerShell (tasto Windows → PowerShell → Invio)
2. Incolla questo comando e premi Invio:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

3. Aspetta che finisca (qualche secondo), poi chiudi e riapri PowerShell.

VERIFICA:

```
uv --version
```

Deve rispondere con qualcosa tipo `uv 0.11.x`.

4

Installare GitHub Desktop

Questo è il programma con cui scarichi il progetto, senza dover imparare i comandi di Git.

1. Vai su desktop.github.com
2. Scarica e installa (doppio clic, poi segue da solo)
3. Alla prima apertura, accedi con l'account GitHub con cui hai accettato l'invito al passo "Prima di iniziare"

5

Scaricare il progetto

1. Apri GitHub Desktop
2. In alto: File → Clone Repository
3. Nella finestra che si apre, vai sulla scheda "GitHub.com" — dovresti vedere il progetto (si chiama wardrobe) nell'elenco. Selezionalo.
4. In basso scegli dove salvarlo sul tuo computer (va benissimo lasciare quello proposto, es. `C:\Users\TuoNome\Documents\GitHub\wardrobe`) e ricorda questo percorso: ti servirà tra un attimo.
5. Clicca Clone.

6

Aprire il terminale nella cartella del progetto

1. Apri Esplora file (l'icona della cartella gialla nella barra in basso) e vai nella cartella dove hai clonato il progetto (passo 5.4)
2. Tieni premuto Shift e fai clic con il tasto destro dentro la cartella (non su un file, nello spazio vuoto)
3. Scegli "Apri finestra PowerShell qui" (o su Windows 11, semplicemente "Apri nel terminale")

Da qui in poi, ogni comando di questa guida va scritto in questa finestra (o in una nuova aperta nello stesso modo, nella stessa cartella).

7

Installare i "pezzi" del progetto (una volta sola)

Nella finestra PowerShell aperta al passo 6, scrivi ed esegui, uno alla volta (aspetta che ognuno finisca prima del successivo — il primo impiega qualche minuto):

```
npm install
```

```
npm run api:sync
```

Se durante `npm install` vedi righe gialle "warning" o messaggi grigi, va tutto bene: sono normali, non sono errori. Un errore vero è scritto in rosso e interrompe il comando.

Provider, API key, e perché paghiamo fal.ai

Prima di scrivere le chiavi al prossimo passo, tre parole che tornano spesso — vale la pena capirle una volta sola.

Provider è l'azienda a cui "affittiamo" il modello di intelligenza artificiale: chi legge le foto e propone gli outfit non è nostro, è un servizio a pagamento a cui Tela si collega. Il progetto ne tiene **quattro intercambiabili**: Anthropic (Claude, quello di riferimento), OpenAI (GPT), Google (Gemini), e uno che gira sul tuo stesso computer senza mandare nulla fuori (Ollama). Sono intercambiabili per costruzione — cambiare provider costa un file di codice, non un mese di lavoro — proprio perché non vogliamo scoprire fra sei mesi di essere legati a uno solo.

API key è una password legata a un account che paga — il nostro, non il tuo — e su cui si accumula la spesa a consumo, un po' come un numero di carta di credito. Da qui le due regole che contano: non finisce **mai** in un file che va su GitHub (per questo vive in `.env`, che il progetto ignora apposta), e si scambia per un canale privato, mai per email in chiaro.

Il progetto chiede al modello **due lavori distinti**, e ogni chiave può servire per l'uno, per l'altro, o per entrambi: **leggere una foto** (capire tipo, colore, tessuto di un capo) e **proporre un outfit** (scegliere cosa abbinare e spiegare perché). Quale provider fa quale lavoro si sceglie dalla schermata **Modelli in uso**, di cui parliamo dopo il Passo 10.

fal.ai non è un provider di linguaggio: è un servizio a parte che serve solo a "staccare" il capo fotografato dallo sfondo, come nelle foto di un catalogo online. Non è indispensabile — se manca quella chiave l'app funziona lo stesso, le foto restano semplicemente con lo sfondo originale. Ma un capo già isolato è anche più facile da leggere per il modello che lo analizza, quindi quando c'è aiuta anche il primo lavoro.

8

Scrivere le chiavi

1. In Esplora file, entra nella cartella `services`, poi `api` (quindi: `wardrobe\services\api`)
2. Trova il file `.env.example` (se non lo vedi, in alto in Esplora file clicca "Visualizza" → spunta "Estensioni nome file" e "Elementi nascosti")
3. Copialo (clic destro → Copia, poi clic destro nello spazio vuoto → Incolla)
4. Rinomina la copia (che si chiamerà tipo `.env.example - Copia`) in `.env` — attenzione: il nome deve essere esattamente `.env`, niente prima e niente dopo, nemmeno `.env.txt`
5. Apri `.env` con il **Blocco Note** (clic destro → Apri con → Blocco note)
6. Vedrai alcune righe che finiscono con `=`, una per ogni chiave possibile (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `GOOGLE_API_KEY`, `FAL_KEY`). **Compila solo le righe per cui hai ricevuto una chiave**, incollandola subito dopo il segno `=` (senza spazi, senza virgolette). Le righe per cui non hai ricevuto nulla lasciale vuote così come sono: l'app funziona comunque, semplicemente quel provider resterà spento finché non gli darai una chiave.
7. Salva (Ctrl+S) e chiudi il Blocco Note.

Questo file resta solo sul tuo computer: non finisce mai su GitHub (è scritto apposta per essere ignorato).

9

Avviare il "cervello" dell'app

Torna alla finestra PowerShell del passo 6 (quella nella cartella principale `wardrobe`, non dentro `services\api`). Scrivi:

```
npm run api:local
```

Dopo qualche secondo deve apparire una riga tipo:

```
API di Tela in ascolto su http://localhost:8787 (DEV_MODE, dati in memoria)
```



Importante: lascia questa finestra aperta. Se la chiudi, l'app smette di funzionare. Puoi ridurla a icona, ma non chiuderla finché stai usando Tela.

10

Avviare l'app

Apri una **seconda** finestra PowerShell nella stessa cartella (ripeti il Passo 6: Shift + clic destro nella cartella `wardrobe` → "Apri finestra PowerShell qui"). In questa nuova finestra scrivi, in ordine:

```
$env:EXPO_PUBLIC_API_URL="http://localhost:8787"
```

```
npm run mobile:web
```

Dopo un po' (la prima volta può metterci un minuto o due) si apre da sola una scheda del browser con l'app. Se non si apre da sola, nel testo che è comparso cerca una riga con scritto `Web is waiting on http://localhost:...` e apri quell'indirizzo a mano nel browser.

Come si usa, in breve

- **Aggiungi** (in basso): fai una foto o scegline una dalla galleria di un capo di vestiario. Aspetta che venga letta — vedrai categoria, colore, tessuto proposti. Se qualcosa è scritto su sfondo corallo/arancione, il modello non ne è sicuro: toccalo per correggerlo. galleria" e scegli una foto che hai già (anche trasferita dal telefono).
 - Se la fotocamera dal browser non si apre o dà errore, usa "Dalla
- **Il tuo armadio**: tutti i capi aggiunti.
- **Chiedi a Tela**: scrivi qualcosa tipo *"Fa caldo, sto per uscire, cosa mi metto?"* e aspetta la risposta.

Provare un provider diverso

Con più di una chiave scritta al Passo 8, puoi scegliere quale modello legge le foto e quale propone gli outfit, senza toccare codice:

1. Nell'app, vai su **Profilo** → **Sviluppo** → **Modelli in uso**
2. Trovi due liste separate — **Lettura foto** e **Suggerimenti** — perché sono i due lavori distinti di cui parlavamo prima del Passo 8, e puoi assegnare un provider diverso a ciascuno
3. **Predefinito del backend** è la prima voce di ogni lista: torna a quello scritto nel `.env`, se mai vuoi annullare la scelta
4. Ogni modello mostra **pronto** o **senza chiave** — è la stessa cosa scritta al Passo 8: senza una riga compilata in `.env`, quel provider resta spento e sceglierlo darebbe un errore

i **Una cosa da sapere:** il `.env` si rilegge solo riavviando la finestra del Passo 9 (`npm run api:local`). Se aggiungi o cambi una chiave, chiudi quella finestra e riesegui il comando prima di aspettarti che l'app la veda.

C'è anche una schermata **Playground modelli**, sempre sotto Sviluppo: è un banco di misura, non una chat — serve a confrontare un modello con un altro su latenza, costo e qualità della risposta, un test alla volta, senza spostare la scelta stabile delle due liste sopra. Lì vedrai anche `gpt-5.1` e `gemini-2.5-pro`: sono i nomi indicati nel documento di design, non ancora confermati sui listini ufficiali — se un provider risponde con un errore sul nome del modello, è il primo posto da controllare, e si corregge scrivendo `MODELLI_OPENAI` o `MODELLI_GOOGLE` nel `.env`, senza toccare codice.

Le prossime volte (versione corta)

Non serve rifare i passi 1-8. Ogni volta che vuoi riaprire Tela:

1. Apri PowerShell nella cartella del progetto (Passo 6), scrivi `npm run api:local`, lascia la finestra aperta
2. Apri una seconda finestra PowerShell nella stessa cartella, scrivi `$env:EXP0_PUBLIC_API_URL="http://localhost:8787" poi npm run mobile:web`

i **Una cosa da sapere:** ogni volta che riavvii il punto 1, l'armadio torna quello di partenza — in questa fase di prova i capi non si salvano in modo permanente da un riavvio all'altro. È normale, non è un errore.

Se qualcosa non funziona

COSA VEDI

COSA FARE

"node non è riconosciuto..."	Node.js non si è installato bene, o serve riaprire PowerShell. Rifai la Verifica del Passo 1.
"python non è riconosciuto..."	Quasi sempre manca la spunta "Add python.exe to PATH": disinstalla e reinstalla Python (Passo 2), spuntando quella casella.
"uv non è riconosciuto..."	Riapri PowerShell (va fatto dopo aver installato uv). Se persiste, rifai il Passo 3.
Il browser dice "Non riesco a raggiungere il server"	Controlla che la finestra del Passo 9 sia ancora aperta e non mostri righe rosse di errore.
La fotocamera non si apre nel browser	Usa "Dalla galleria" invece di "Scatta", e scegli una foto già esistente.
Le foto restano con lo sfondo originale	Normale se non hai messo <code>FAL_KEY</code> al Passo 8 (è facoltativa) — l'app funziona lo stesso.
Un provider risulta "senza chiave" in Modelli in uso o nel Playground	Manca (o è sbagliata) la riga corrispondente nel <code>.env</code> — vedi Passo 8 — oppure hai dimenticato di riavviare <code>npm run api:local</code> dopo averla scritta.

Se un errore non è in questa tabella, copia il testo esatto (anche una foto/screenshot va benissimo) e mandalo a chi ti ha dato questa guida.